DeCal: Decoupled Tail Budget Control for Non-Stationary LLM Serving

Anonymous Author(s)

Abstract

Cloud-hosted LLM services must commit capacity budgets before the next request window reveals its tail demand. Our characterization of the public ServeGen and BurstGPT traces shows frequent shifts in window-level P99 token demand, which can make budgets derived from completed history stale. Error-driven calibration then reacts only after an affected window has finished. A temporary budget increase can protect subsequent windows once drift evidence becomes available, but scoring later errors against the protected budget can reduce the recorded residual and weaken future corrections. Tail budgeting must therefore separate near-term capacity protection from the reference used for long-term error learning. To this end, we present **DeCal**, a tail budget controller that composes each budget from a base estimate, a bounded and retractable protection term, and a calibration correction. After drift evidence enters completed history, a lightweight trigger activates protection for the earliest subsequent window, while a state machine holds and then retracts this protection. In parallel, the calibrator scores realized P99 demand against the unprotected base estimate rather than the emitted budget, keeping short-term protection separate from the long-term learning target. Trace-driven evaluation demonstrates that DeCal comprehensively outperforms 12 state-of-the-art baselines across all indicators. Specifically, on ServeGen and BurstGPT traces, DeCal reduces P99 latency by 1.0% and 0.5% while increasing SLO Goodput by 1.9% and 0.7%, respectively, without sacrificing token throughput or Quality Proxy indicators. Overall, DeCal delivers a robust and auditable upstream capacity signal that effectively tames demand non-stationarity in LLM services.

CCS Concepts: • **Software and its engineering** → **Cloud computing**; • **Computing methodologies** → *Machine learning*.

Keywords: LLM serving, workload characterization, KV-cache budgeting, tail drift

ACM Reference Format:
Anonymous Author(s). 2026. DeCal: Decoupled Tail Budget Control for Non-Stationary LLM Serving. In *Proceedings of ACM Conference (Conference'17).* ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Cloud-hosted LLM services must serve interactive requests with highly variable output lengths. Since KV-cache demand grows with processed sequence length, tail-length variation directly affects queueing, latency, and memory pressure [12, 16, 18, 21]. Production workload studies further show that request rates and token lengths vary substantially over time [22, 24]. The serving stack must therefore reserve enough near-term capacity before demand is fully visible, while avoiding reservations that waste GPU resources [28]. This trade-off is time constrained. At window granularity, the relevant budget must be committed before the next window reveals its tail demand. The visibility boundary turns workload characterization from descriptive measurement into a budget-control problem.

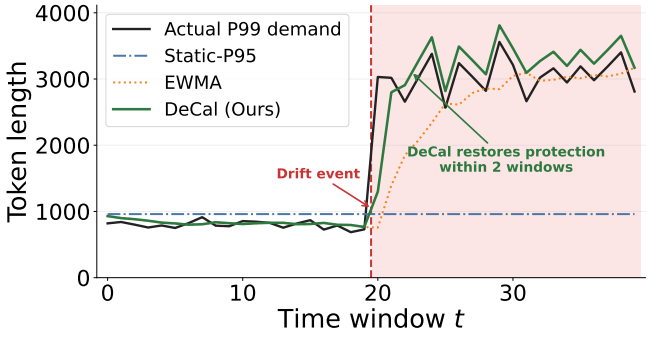


Figure 1. Visibility boundary for next-window tail budget control around a drift. Budgets are committed before the target-window tail is observed, and feedback can affect only later windows.

We study tail-length proxy budgeting for the next window. Before each target window, a controller emits one scalar tail-length proxy budget for each budget entity using only completed history. This scalar acts as an upstream capacity signal that primarily sets the admission envelope and can also be consumed by quota or scaling policies. The design goal is to track window-level P99 tail demand in the same token coordinate under non-stationary workloads. Because that demand is not yet visible at commitment time, all decisions must be based on closed windows.

Prior methods for tail estimation, workload forecasting, and online calibration provide complementary but incomplete responses to this visibility boundary. A completed-history baseline derives the next budget from closed windows, while historical high-percentile summaries retrospectively describe observed tail levels. Workload forecasters project past workload context into the next window [6, 26]. Online conformal methods play a different role by correcting budgets after prior coverage errors are observed [2, 9, 10].

These roles occupy different points in the budgeting pipeline. Forecasting refines the pre-window estimate, whereas online calibration adjusts coverage from previously observed errors. Both, however, depend on evidence that has already entered completed history. After an abrupt upward shift, forecasting still extrapolates from pre-shift context, and error-driven calibration must wait until a drifted residual is observed [17].

The resulting exposure is shown in Fig. 1. A budget calibrated before an upward drift remains below the realized P99 after the boundary, and correction becomes possible only after the drifted window has completed. A temporary budget increase can reduce the exposure once drift evidence enters completed history. The remaining difficulty is feedback contamination. Scoring later errors against a protection-increased emitted budget can reduce the recorded residual and weaken future corrections. A controller must therefore protect near-term risk windows while preserving a base-budget residual signal for long-term calibration.

Challenges. Tail budget control is constrained by three causal control constraints. **C1: visibility lag.** Output lengths and their resource requirements are not known at request arrival; at the coarser control granularity studied here, the target window likewise reveals its tail only after its budget has been emitted. The controller can therefore use only completed windows and cannot tune against hindsight P99 values. **C2: asymmetric cost.** Missing an upward drift creates budget-miss, queueing, or preemption pressure, whereas persistent reservation or over-provisioning reduces utilization and cost efficiency. The controller must therefore protect risk windows and then retract. **C3: feedback contamination.** If a calibrator scores the realized tail against a protection-increased emitted budget, successful protection can be recorded as over-allocation and depress later corrections. A deployable controller must raise the current budget without making that action the later calibration-score reference. These constraints lead to the central question. *What control structure allows post-drift protection and long-term calibration to coexist under completed-history visibility?*

Approach. DeCal answers this question by decomposing budget generation into three roles. A base estimator predicts the next-window tail level from completed windows. A drift trigger and protection state machine add a bounded, retractable increase as soon as drift evidence becomes causally visible. A decoupled calibrator updates its residual buffer against the unprotected base budget, not the protected emitted budget. This order lets temporary protection cover short post-drift exposure without becoming evidence that the base budget was too high. When local history is thin or stale after severe drift, DeCal may initialize or recover the base estimator with a transfer prior from similar completed-history entities, but transfer does not affect the protection path or the calibration reference.

Contributions. We make three contributions to workload characterization and budget control for LLM serving.

- We identify an action-reference contamination failure mode in completed-history LLM serving control: a temporary protected budget can corrupt the residual sequence used for later calibration if the emitted action becomes the score reference.
- We identify a calibration failure mode caused by using the protected emitted budget as the later error reference. Scoring against such a budget lets a controller learn away its own protection. DeCal avoids this failure by scoring residuals against the unprotected base budget.
- We evaluate DeCal with public LLM-serving traces under causal replay and show that the decoupled reference improves direct budget reliability, missed-tail mass, and replay-derived capacity-pressure indicators over close online-calibration baselines.

Artifact availability. Our code is available on https://anonymous.4open.science/r/DeCal-B2B2.

2 Problem Background and Motivation

2.1 KV-Cache Pressure and Tail Budget Interface

For a deployed LLM, KV-cache demand scales monotonically with sequence length. A request of processed length ℓ consumes approximately $2LH_{kv}db\ell$ bytes, where L is the number of layers, H_{kv} is the number of KV heads, d is the head dimension, and b is bytes per scalar. Because the next window's tail demand is not yet visible when capacity decisions are made, the serving stack needs an upstream capacity signal before the window starts. The controller output is therefore not a KV-block allocation plan, but a token-coordinate signal that can be consumed by downstream serving policies.

We instantiate this signal as a proxy in the same token coordinate. We denote by $B_{c,t}$ the scalar tail-length proxy budget for budget entity c in window t, measured in tokens. Each completed window has an empirical token-length distribution $P_{c,t}$, and $B_{c,t}$ is compared with the empirical quantile $q_\alpha(P_{c,t})$ in that coordinate. Because KV-cache bytes are monotone in token length for the same deployed model, $B_{c,t}$ is a window-level proxy for KV-cache pressure. A P99 budget miss occurs when $q_{0.99}(P_{c,t}) > B_{c,t}$, meaning that the upstream budget does not cover the window-level P99 tail demand. The miss records proxy capacity risk rather than a physical per-request OOM in a particular engine.

At the serving boundary, the proxy budget primarily sets a next-window admission limit. The same scalar can also be consumed by quota or scaling policies without making DeCal an execution-layer scheduler. After each completed window, the serving layer provides an entity id, timestamp, request count, and token-length histogram. Before the next window, DeCal returns a tail-length proxy budget in the same token coordinate. Request scheduling and KV-block allocation remain downstream consumers of that budget.

The interface yields a completed-history budgeting problem. For entity c and window t, $P_{c,t}$ is observable only after the window closes. A controller must emit $B_{c,t}$ before window t starts, using only completed history from windows $0, \cdots, t-1$. The same budget-demand comparison defines both risk and cost. The primary risk metric is the P99 budget-miss rate, the fraction of windows where $q_{0.99}(P_{c,t}) > B_{c,t}$. The cost metric is mean over-allocation, $\max(0, B_{c,t} - q_{0.99}(P_{c,t}))$. Tail under-allocation appears immediately as budget-miss risk, whereas over-allocation consumes shared capacity.

2.2 Workload Characterization: Three Observations

We characterize workload properties that constrain a causal budget controller. For ServeGen [24] and BurstGPT [22], each budget entity has a completed-history sequence of empirical token-length distributions $P_{c,t}$. From these sequences, we compute P50, P95, P99, heavy-tail mass, and adjacent-window Jensen–Shannon divergence $\mathrm{JS}(P_{c,t-1}, P_{c,t})$. A ServeGen entity corresponds to a client, whereas a BurstGPT entity corresponds to a stream, so we avoid treating BurstGPT streams as true tenants. The observations below serve as design constraints rather than descriptive statistics alone. O1 establishes the need for continuous completed-history control. O2 shows why protection and calibration require separate time scales. O3 bounds cross-entity transfer to startup and recovery priors rather than global pooling.

O1: Persistent tail drift prevents reliance on a stable calibration period. Fig. 2 shows that P99 token demand repeatedly moves between tail states rather than fluctuating around one stationary level. With drift defined as adjacent-window JS ≥ 0.1, 42.2% of ServeGen language m-mid transitions and 89.4% of BurstGPT transitions are drift events. The cross-entity interquartile range (IQR) also expands and contracts over time, so a global safety margin would mask entity-level risk. Budget control therefore cannot rely on one stable calibration period. It must operate continuously on completed history and protect later windows before enough post-drift failure feedback has entered the buffer. Appendix D, especially Figs. 16 and 17, reports sensitivity analyses showing that this conclusion is not an artifact of a single JS threshold, window granularity, or minimum-request filter.

O2: Drift severity is graded rather than binary. Fig. 3 shows that drift is not binary. The JS spectrum contains calm, mild, medium, and severe transitions, and medium-to-severe drift recurs in both workload sources. BurstGPT has a larger severe-transition mass, while ServeGen still shows substantial medium drift under 10-minute windows. The spectrum rules out a single global response. A low threshold would keep protection active too often, while a high threshold would miss medium-severity mass. The controller therefore needs a fast protection path for exposed windows and a slower calibration path with a stable calibration reference.

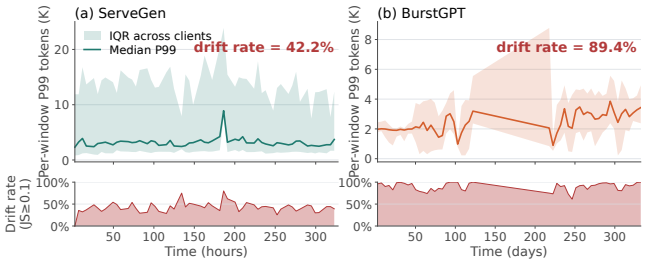


Figure 2. Window-level P99 token-length trajectories and rolling drift-rate tracks.

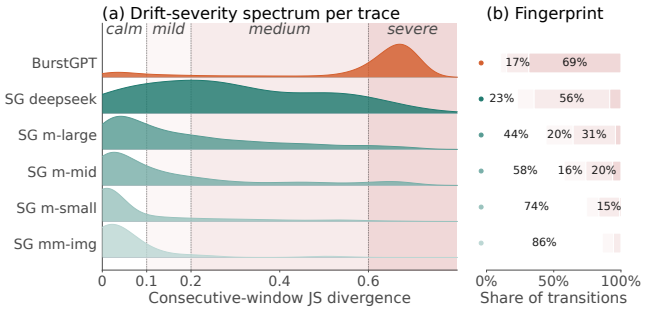


Figure 3. Adjacent-window JS-divergence spectra and severity bins. Panel (a) plots per-trace divergence distributions with threshold guides. Panel (b) groups transitions into calm, mild, medium, and severe intervals. SG denotes ServeGen.

O3: Similar entities provide bounded recovery priors. Fig. 4 shows visible similarity blocks among ServeGen clients, while the four BurstGPT streams serve as an independent production-style check rather than evidence for large-scale tenant clustering. The observed heterogeneity cautions against assuming one pooled tail state, whereas the similarity blocks motivate testing whether nearby entities can help when local history is thin or stale after severe drift. DeCal therefore uses cross-entity information only as a bounded prior built from completed history. It may initialize or recover the base estimator, while local history and the calibration reference remain the authoritative update path. The evaluation establishes the benefit of selective transfer over local-only cold start. Global pooling is outside the claim supported by this evidence.

Together, these observations make non-stationarity a control constraint. First, persistent tail-state changes make static tail quantiles and one-time calibration insufficient for runtime budgets. Second, drift spans multiple severity levels, so the system needs fast protection for current risk while keeping that protection out of later error learning. Third, entity similarity may help when local evidence is scarce, but the observed heterogeneity requires transfer to remain bounded and subordinate to causal local history. The next

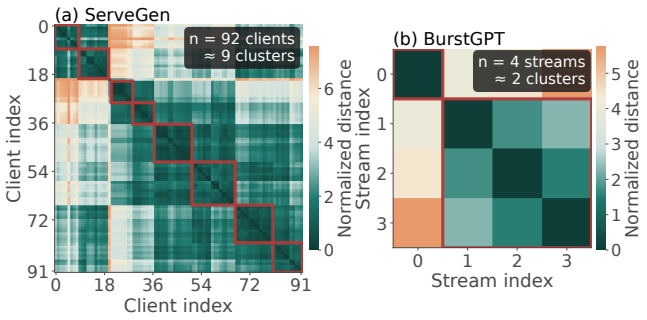


Figure 4. Budget-entity similarity matrices computed from normalized workload summaries. Darker cells indicate lower distance.

section shows why directly coupling protection and calibration creates a feedback-level failure mode, and then derives DeCal's decoupling principle.

2.3 Motivation

The preceding observations make the central difficulty a feedback-attribution problem, not merely a tail-prediction problem. After upward drift becomes visible, a controller must raise the near-term budget quickly enough to avoid under-budgeting, yet it must also retract that temporary margin once the exposure period has passed. These requirements separate the roles of the available feedback signals. Drift evidence indicates when the current base estimate may be stale. Calibration estimates residual budget error after a window has completed. Cross-entity similarity can provide a prior when local history is limited, but it cannot replace the local error sequence. The design question is how to let protection act immediately while preserving a calibration target that remains independent of that action.

The need for separation follows from the temporal order of completed-history control. The controller first commits the budget for window t, observes $q_{0.99}(P_{c,t})$ only after the window closes, and then updates calibration from the completed window. A drift-triggered increase in the emitted budget is an action taken before observation, not evidence that the base estimator has become accurate. Coupling arises when the subsequent calibration step uses this action-augmented budget as its reference. The protection margin is then subtracted from the observed error, and a successfully protected window contributes a smaller calibration score. Stronger current protection can therefore make later residual learning appear less necessary, even when the realized tail demand remains high.

The resulting failure mode is a coupling trap in the calibration reference. The problem is not that drift protection is too weak or that calibration is too slow in isolation. The problem is that the same emitted budget is used both as the current-window control action and as the statistical reference for learning long-run calibration error. Let $B^{\text{base}}_{c,t}$ denote the base budget before protection, $\gamma_{c,t} \geq 1$ the drift-triggered protection coefficient, and $C_{c,t}$ the additive calibration correction. A coupled controller emits $B_{c,t} = B^{\text{base}}_{c,t}\gamma_{c,t} + C_{c,t}$ and later scores calibration against this full emitted budget. The calibration score then exhibits the Coupling Trap.

Proposition 1. Under the coupled calibration score

$$s_{c,t} = q_{0.99}(P_{c,t}) - B_{c,t}, \tag{1}$$

the score changes with the protection coefficient as

$$\frac{\partial s_{c,t}}{\partial \gamma_{c,t}} = -B^{\text{base}}_{c,t} \leq 0. \tag{2}$$

The derivative follows directly by substituting the coupled emitted budget defined above while holding the realized tail, base budget, and additive correction fixed. Hence the recorded calibration score is monotone non-increasing in the protection coefficient. Any monotone correction rule based on recent scores, including empirical quantiles, or exponential updates, cannot increase its later correction in response to a larger protection coefficient. When $B^{\text{base}}_{c,t} > 0$, stronger protection is recorded as a smaller error, so a coupled controller can weaken its later correction even when the realized tail is unchanged.

Proposition 1 locates the problem in the calibration reference. Changing the drift threshold, protection magnitude, or calibration step size does not remove the coupling. Reducing the protection coefficient can shrink the one-step effect but cannot change the derivative sign. Speeding up the calibrator only makes it absorb the action-distorted score faster. Any controller that first expands the budget and then scores against the expanded budget faces an equivalent coupling risk.

DeCal resolves the trap by decoupling the calibration reference from the protected emitted budget. The protection path may still change the budget committed for the current window. The calibration path, however, learns the base-budget error and therefore scores against the base budget before protection.

$$s_{c,t} = q_{0.99}(P_{c,t}) - B^{\text{base}}_{c,t}. \tag{3}$$

Then $\gamma_{c,t} \notin s_{c,t}$, equivalently $\partial s_{c,t}/\partial \gamma_{c,t} = 0$. The current-window budget can be protected without changing the meaning of later calibration scores. The relevant design requirement is not simply a larger post-drift budget, but a calibration error sequence tied to an unchanged learning target. DeCal is designed around this invariant, and it allows different base estimators, drift detectors, and protection actions. The required invariant is that the action protecting the current window must not become the reference for learning base-budget error.

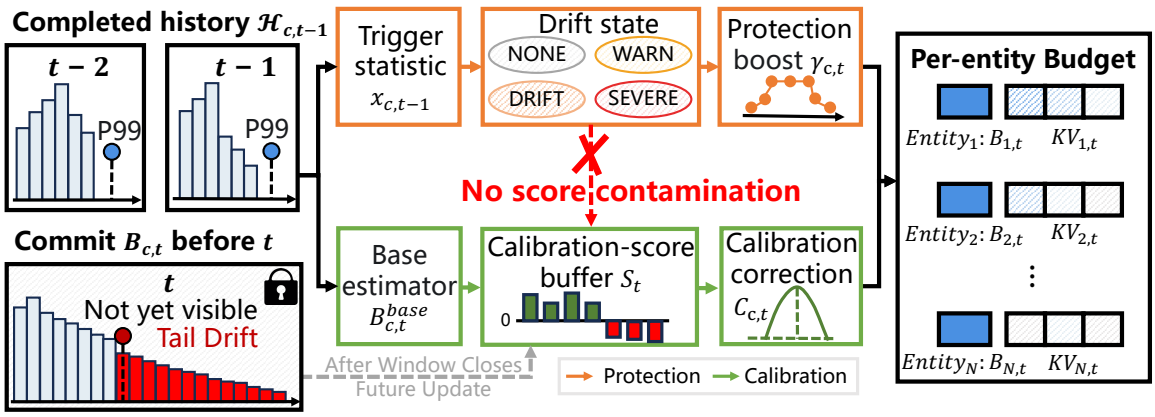


Figure 5. DeCal control state and causal data flow. Solid arrows denote information available before budget emission. Post-window observations update the base estimator, drift state, and calibration buffer only after the window closes.

3 DeCal Design

DeCal instantiates the completed-history interface in Section 2.1 while addressing the Coupling Trap in Section 2.3. The controller follows a causal contract. No budget decision for window t may use the window-level P99 of that window, which is observed only after the budget has been committed. Its state is therefore limited to completed local history and committed cross-entity information. DeCal satisfies this contract by assigning budget generation to three roles with separate feedback semantics. The base estimator tracks the recent tail level, the protection path covers the short exposure period after drift evidence first enters completed history, and the calibration path learns only the residual between the base budget and the window-level P99. Fig. 5 shows the corresponding causal data flow, with separate protection and calibration paths around the base estimator. Two invariants enforce the separation. Protection may change the current emitted budget, but the protected action must not become the later calibration reference. Selective transfer may provide a completed-history prior when local evidence is weak, but it affects only the base estimator and leaves the protection–calibration boundary unchanged.

3.1 Control Interface and Decoupling Invariant

DeCal is a window-level budget controller that sits upstream of an LLM serving engine. It does not replace the scheduler or the KV-cache allocator. Instead, before each window starts, it emits a scalar tail-length proxy budget $B_{c,t}$ for budget entity c. A budget entity denotes the deployment unit for which the serving stack maintains a separate capacity commitment. DeCal requires only an entity identifier and the empirical token-length distribution $P_{c,t}$ after the window closes, so the same interface applies to tenants, queues, or streams.

Let $\mathcal{H}_{c,t-1} = \{P_{c,0}, \cdots, P_{c,t-1}\}$ denote the completed history visible to entity c before window t starts. The interface contract is causal. $B_{c,t}$ can depend only on $\mathcal{H}_{c,t-1}$ and on cross-entity state that has been completed and committed. The realized window-level P99, $q_{0.99}(P_{c,t})$, can enter the controller state only after window t closes. Every update therefore follows the same commit, observe, and update order.

Under this contract, treating the next-window budget as one indivisible prediction would mix three different feedback roles. A single budget term would have to track the recent tail level, cover the short risk period after drift becomes visible, and learn the persistent residual between the base estimator and the window-level P99. DeCal instead assigns these roles to separate terms. Fig. 5 shows the resulting decomposition. Completed history drives the base tail estimator, drift trigger, and calibration correction, while the current window's P99 can be written back only after the window closes. The emitted budget can be written in equivalent multiplicative and additive forms.

$$\begin{aligned} B_{c,t} &= B^{\text{base}}_{c,t}\gamma_{c,t} + C_{c,t} \\ &= B^{\text{base}}_{c,t} + (\gamma_{c,t} - 1)B^{\text{base}}_{c,t} + C_{c,t}. \end{aligned} \tag{4}$$

Here $B^{\text{base}}_{c,t}$ is the base budget, $\gamma_{c,t} \geq 1$ is a retractable protection coefficient, and $C_{c,t}$ is the calibration correction. The base estimator supplies the recent tail level, the protection path acts only through $\gamma_{c,t}$ to cover short post-drift exposure, and the calibration path acts only through $C_{c,t}$ to learn the residual between the base budget and the window-level P99. The decomposition also defines the control boundary. The protection path can change the emitted budget, but the calibration path must always score against the base budget before protection and before the already emitted calibration correction.

Algorithm 1 makes the per-window order explicit. At the start of a window, the controller reads drift statistics from completed history and produces an alarm $a_{c,t}$. The protection state machine updates $\gamma_{c,t}$ from that alarm. If the entity is in cold start or severe-drift recovery, the controller may inject a transfer prior from the completed-and-committed transfer

Algorithm 1: Completed-History DeCal Window Update

Input: $\mathcal{H}_{c,t-1}$, committed pool $\mathcal{T}_{t-1}$, $\hat{P}_{c,t-1}$, drift-trigger and protection states, and $\mathcal{S}_{c,t-1}$.
Before window t
1. Compute $x_{c,t-1}$ and alarm $a_{c,t}$ from completed history.
2. Update and retract $\gamma_{c,t}$ after the hold expires.
3. Form an optional completed-history prior from $\mathcal{T}_{t-1}$.
4. Compute $B_{c,t}^{\text{base}}$ and $C_{c,t}$.
5. Emit $B_{c,t}$.
After window t
6. Observe $P_{c,t}$ and update base/drift state.
7. Append base-budget score $s_{c,t}$, excluding emitted $B_{c,t}$.
8. Commit completed entities to $\mathcal{T}_t$.

pool to initialize the base estimator. The calibration path then computes $C_{c,t}$ from historical calibration scores and emits $B_{c,t}$. Only after the window closes does the controller observe $P_{c,t}$ and $q_{0.99}(P_{c,t})$, update the base estimator and calibration buffer, and cache drift statistics for the next window. For entities sharing a timestamp, the transfer pool is committed only after all entities at that timestamp have been processed, which prevents within-window cross-entity leakage.

3.2 Fast Protection Path after Drift

The protection path covers the interval between first drift evidence and reliable post-drift calibration. If tail demand shifts inside window t, the controller has already committed $B_{c,t}$ and cannot use that shift until the window closes. The first budget that can react is for window t+1, but the calibration buffer may still be dominated by pre-drift residuals. Relying only on calibration exposes the first post-drift windows to under-budgeting. Keeping a permanently conservative budget solves that exposure by creating persistent waste. DeCal separates these concerns through a bounded protection coefficient $\gamma_{c,t}$ that is explicitly retracted after the exposure interval. Algorithm 1 places this protection update before the base budget and calibration correction are emitted.

The trigger supplies timing evidence rather than a budget magnitude. For budget entity c, the trigger input is

$$x_{c,t-1} = \varphi(P_{c,t-1}), \tag{5}$$

where $\varphi(\cdot)$ is the window-level mean length by default and can also be set to the P95 length. The input is computed only from completed windows. The trigger indicates when the current base estimate may be stale. It does not predict the next-window P99 or determine the final budget. The resulting separation keeps drift timing independent of magnitude estimation and of the calibration-reference invariant. A stronger detector can replace this trigger without changing the base estimator or the calibration path.

The default trigger uses a Page–Hinkley statistic. Let $\hat{\mu}_{c,t-1}$ and $\hat{\sigma}_{c,t-1}$ denote completed-history location and scale estimates before window t. We use the sign convention

$$\begin{aligned} z_{c,t} &= \frac{\hat{\mu}_{c,t-1} - x_{c,t-1}}{\max(\hat{\sigma}_{c,t-1}, \epsilon)}, \\ m_{c,t} &= m_{c,t-1} + z_{c,t} - \delta, \\ M_{c,t} &= \max\{M_{c,t-1}, m_{c,t}\}, \\ \mathrm{PH}_{c,t} &= M_{c,t} - m_{c,t}. \end{aligned} \tag{6}$$

An upward change in the completed-window statistic makes $z_{c,t}$ negative, decreases $m_{c,t}$, and increases $\mathrm{PH}_{c,t}$ relative to its running maximum. The alarm scale λ is applied to this statistic: WARN is emitted at 0.7λ, DRIFT at λ, and SEVERE at 2λ. The evaluated default fixes this operating scale at $\lambda = 5.0$ and uses $\delta = 0.3$. These thresholds only set the protection operating point. They do not define calibration scores or the long-term coverage target.

The alarm is a control signal for temporary protection. A protection state machine maps alarms to a bounded coefficient. In the normal state, $\gamma_{c,t} = 1$. By default, WARN, DRIFT, and SEVERE use $\gamma_{c,t} = 1.1$, 1.2, and 1.4. A WARN alarm is held for at least one window, while a SEVERE alarm initializes a hold counter to $h = 3$. Section 4.5 varies both the trigger thresholds and the protection coefficients to test the budget-miss–waste frontier. For DRIFT and SEVERE alarms, DeCal also resets the calibration buffer and seeds it with a non-negative tail-gap score. The seed prevents calibration corrections from relying on stale residuals. Clearing the buffer alone can temporarily collapse the correction to zero, whereas retaining the old buffer prolongs stale-state bias. Reset plus seeding gives the calibration path a conservative restart without making protection part of the later score reference.

The hold counter keeps protection active while post-drift calibration evidence accumulates. During the hold window, the protection coefficient does not immediately fall back even if subsequent alarms return to NONE. After the hold counter expires, DeCal retracts the coefficient smoothly.

$$\gamma_{c,t+1} = 1 + (\gamma_{c,t} - 1)(1 - \rho), \tag{7}$$

where $\rho \in (0, 1]$ controls the retraction speed. Fig. 6 summarizes the temporal rationale for making protection temporary. The relative window indices show that post-shift evidence is observed only after the shifted window closes, while the calibration-score buffer remains dominated by pre-shift scores for the first affected budgets. DeCal therefore boosts $\gamma_{c,t}$ only during this calibration-exposed interval and retracts it once the hold period expires. This timing separation lets protection cover early post-drift risk without turning the protection margin into a persistent calibration target.

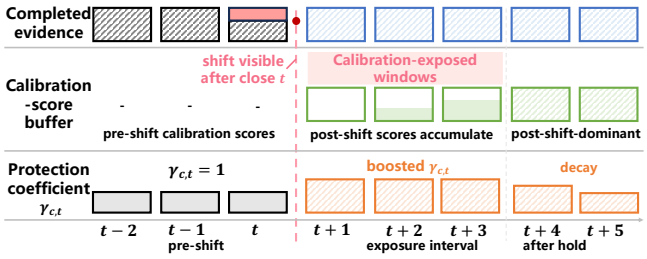


Figure 6. Timing role of the temporary protection path. The window labels are relative indices, not physical time values. Post-shift evidence becomes available only after the shifted window closes, while the calibration buffer needs additional post-shift scores before it can dominate the correction. DeCal boosts $\gamma_{c,t}$ during this calibration-exposed interval and retracts it after the hold period.

3.3 Decoupled Calibration against the Base Budget

The calibration path is defined by the budget against which post-window errors are scored. DeCal uses the unprotected base budget as this reference and treats protection as current-window actuation. The resulting order separates the correction emitted before window t from the score written after that window closes. Algorithm 1 follows this order by computing $C_{c,t}$ from past residual scores before window t starts and appending a new score only after $P_{c,t}$ is observed. For budget entity c, DeCal maintains a smoothed distribution $\hat{P}_{c,t-1}$ over completed history and uses a base quantile β to produce the base budget.

$$B^{\text{base}}_{c,t} = q_\beta(\hat{P}_{c,t-1}). \tag{8}$$

The default is $\beta = 0.95$. The base quantile gives the estimator a tracking role rather than a final-coverage role. A much higher base quantile would improve short-term coverage but would push steady-state windows toward over-allocation. A much lower base quantile would force calibration to absorb routine tail mass. DeCal therefore assigns recent-tail tracking to the base estimator and reserves upper-tail residual correction for calibration.

Using this base reference, the pre-window step computes the correction from past base-budget residuals. Let $n = |\mathcal{S}_{c,t-1}|$. When $n \geq 5$, the correction is the non-negative empirical quantile of those residuals

$$\begin{aligned} C_{c,t} &= \max\big(0,\ Q_{\alpha_n}(\mathcal{S}_{c,t-1})\big), \\ \alpha_n &= \min(\tau(1+1/n),\ 1), \end{aligned} \tag{9}$$

where τ is the target coverage level, defaulting to $\tau = 0.95$. The buffer stores at most the most recent R calibration scores, with default $R = 100$. When fewer than five scores are available, a direct empirical correction is too unstable. Setting $C_{c,t} = 0$ exposes tail risk after cold start or drift reset, while inheriting an old buffer carries stale-state bias into the new regime. DeCal instead seeds the buffer with the non-negative smoothed tail gap $q_{0.99}(\hat{P}_{c,t-1}) - q_\beta(\hat{P}_{c,t-1})$. The seed gives the calibration path an initial correction tied to the current base estimate until enough new scores are available.

The post-window score then determines whether calibration learns a base-budget residual or the effect of the controller's own action. Fig. 7 contrasts the relevant score references. Fig. 7(a) shows the intended base-budget residual between $B^{\text{base}}_{c,t}$ and the realized window-level P99. The residual excludes both the protection coefficient and the correction already emitted for the current window. Including either term would make the calibrator learn the effect of its own action rather than the error of the base estimator. If the controller scores against the full emitted budget after window t closes, the score is

$$\begin{aligned} \tilde{s}_{c,t} &= q_{0.99}(P_{c,t}) - B_{c,t} \\ &= q_{0.99}(P_{c,t}) - \big(B^{\text{base}}_{c,t}\gamma_{c,t} + C_{c,t}\big). \end{aligned} \tag{10}$$

Fig. 7(b) shows the coupled full-budget reference. Once $\gamma_{c,t}$ and the existing correction enter the reference budget, a stronger current action lowers $\tilde{s}_{c,t}$ even when the realized P99 is unchanged. The next correction can then shrink for the wrong reason. The same self-reference arises when $C_{c,t}$ is included, because the calibrator may treat its own previous correction as evidence that less residual remains. Successful protection and prior correction are then recorded as apparent over-allocation, which is the Coupling Trap from Section 2.3.

DeCal avoids this coupling by writing only the base-budget residual.

$$s_{c,t} = q_{0.99}(P_{c,t}) - B^{\text{base}}_{c,t}. \tag{11}$$

The score enters $\mathcal{S}_{c,t}$ after the window closes, and its reference budget contains neither $\gamma_{c,t}$ nor $C_{c,t}$. Hence $\partial s_{c,t}/\partial \gamma_{c,t} = 0$. Fig. 7(c) shows this base-budget score reference. DeCal still uses the full emitted budget to protect the current window, but it uses only the base budget as the reference for later learning. The protection coefficient cannot mechanically depress future calibration corrections, and an existing correction is not reinterpreted as a new error reference. The resulting separation keeps current actuation out of the future learning target.

The invariant is structural rather than estimator-specific. As long as the current-window action is not used as the later calibration-score reference, protection cannot mechanically weaken calibration in reverse. The base estimator and Page–Hinkley trigger are lightweight instantiations of this invariant. A stronger predictor can replace the base estimator, and a different detector can replace the alarm, provided that the emitted action is not written into the calibration score. The full-action-reference ablation in Section 4.3 evaluates the broader reference choice. Because the base reference excludes both protection and the previously emitted correction, the ablation supports action–reference separation but does not attribute the entire gap to either excluded term alone.

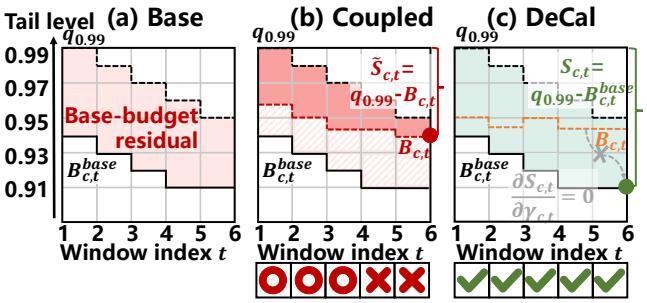


Figure 7. Calibration-reference choices for decoupled residual learning. DeCal uses the base-budget reference to keep temporary protection outside the learning target.

3.4 Selective Transfer Prior

Selective transfer supplies a prior only for the base estimator when local completed history is sparse or likely stale. It is used during cold start and severe-drift recovery, not as a calibration mechanism. At cold start, DeCal considers transfer only after the first local distribution has been committed, giving the target entity a causal reference for similarity matching. During normal operation, transfer becomes eligible only after a SEVERE alarm indicates that the local base estimator may no longer represent the current regime. Otherwise, the base estimator is updated from the entity's own completed history. The eligibility rule makes cross-entity evidence an initialization or recovery prior rather than an alternative feedback path.

The donor set is restricted to committed snapshots from completed windows. For target entity c, a donor snapshot from entity j is eligible only if it has been committed before the current budget is emitted. DeCal ranks eligible donors by $W_1(F_c, F_j)$, the Wasserstein-1 distance between empirical cumulative distribution functions (CDFs). Here F_c is the CDF of the target entity's current local reference distribution, and F_j is the CDF of donor snapshot P_j. Let $\mathcal{N}_k(c)$ denote the k nearest eligible donors, with default $k = 3$. DeCal forms the weighted prior

$$\tilde{P}_c = \frac{\sum_{j\in\mathcal{N}_k(c)} w_j P_j}{\sum_{j\in\mathcal{N}_k(c)} w_j}, \qquad w_j = \frac{1}{1 + W_1(F_c, F_j)}. \tag{12}$$

The inverse-distance weights favor nearby tail distributions while keeping donor influence bounded. The resulting prior initializes or recovers the smoothed distribution that produces $B^{\text{base}}_{c,t}$ before the current budget is emitted.

Transfer does not modify any state used by the protection or calibration paths. It leaves the protection coefficient, the calibration-score buffer, and the score reference unchanged. When multiple entities share a timestamp, all budget emissions for that timestamp are completed before any current distributions are admitted to the transfer pool. The batch-commit rule prevents current-window information from leaking across entities. Transfer changes only the starting point of the base estimator, while the protection–calibration boundary remains fixed.

Because transfer is invoked only on startup or recovery paths, it remains outside the normal per-window update. Appendix B gives the online-cost accounting. The startup-and-recovery placement preserves a local completed-history update path during steady operation.

4 Evaluation

We evaluate DeCal as an upstream tail budget controller under completed-history replay. Completed-history replay is stricter than offline prediction, because each method must commit a next-window tail-length proxy budget before the target window is revealed. All methods are then scored by the same capacity-pressure evaluator. Direct metrics measure P99 budget misses, request-weighted miss mass, and waste. Throughput, latency, Quality Proxy, and SLO Goodput are deterministic post-emission indicators derived from the same budget sequence and realized histograms.

The evaluation is organized around five questions. **RQ1: Overall Performance** evaluates capacity-pressure behavior and budget-demand alignment. **RQ2: Ablation Study** isolates base-budget scoring, protection, and transfer. **RQ3: Trigger Behavior** examines trigger timing on completed-window changes. **RQ4: Sensitivity and Cost** studies protection strength, hold length, coverage target, cold-start transfer, and online overhead. **RQ5: Capacity-Signal Utility** evaluates downstream KV-pool sizing and selected-entity capacity analysis. Appendix C provides metric definitions and expanded setup details. Appendix D reports stress baselines, paired-bootstrap uncertainty, drift-threshold sensitivity, preprocessing sensitivity, and workload diagnostics.

4.1 Experimental Setup

Replay protocol. The evaluation unit is a budget-entity window with empirical token-length distribution $P_{c,t}$. Before window t starts, each controller emits $B_{c,t}$ using only $\mathcal{H}_{c,t-1} = \{P_{c,0}, \cdots, P_{c,t-1}\}$ and completed-and-committed cross-entity state. The realized distribution $P_{c,t}$ and target $y_{c,t} = q_{0.99}(P_{c,t})$ are revealed only after the window closes. The evaluator then computes failures, waste, tail quality, and replay-derived capacity-pressure indicators before submitting the completed window to the controller update interface. All methods share preprocessing, windowing, entity order, replay seeds, and metric computation, so only the budget-decision module differs.

Table 1. Replay-derived capacity-pressure metrics. Values are mean ± standard deviation; **bold** marks the best result and underline marks the second-best result.

Method	ServeGen					BurstGPT				
	Throughput (token/s) ↑	P95 Latency (ms) ↓	P99 Latency (ms) ↓	Quality Proxy (%) ↑	SLO Goodput (token/s) ↑	Throughput (token/s) ↑	P95 Latency (ms) ↓	P99 Latency (ms) ↓	Quality Proxy (%) ↑	SLO Goodput (token/s) ↑
VLLM-LTR	20723.79±11.98	3728.71±7.98	4557.45±3.22	45.10±0.01	18936.41±22.17	4091.16±16.07	3606.72±40.11	4589.62±8.53	44.95±0.05	3783.70±22.36
EGTP	20720.48±8.85	3723.70±4.24	4563.06±0.93	45.12±0.02	18941.09±13.85	4162.21±10.12	3254.53±29.68	4515.73±6.66	45.35±0.02	3917.96±14.05
Fremer	20627.75±93.57	3804.56±54.40	4582.71±21.34	44.98±0.07	18705.46±188.95	4217.83±2.84	2860.91±9.30	4424.98±4.39	45.66±0.01	4024.25±3.62
OrgLinear	20682.66±51.41	3761.28±42.90	4575.74±18.83	45.07±0.06	18837.14±126.83	4181.87±10.54	3085.62±96.23	4470.12±18.83	45.48±0.08	3960.81±22.20
PRISM	20712.59±27.69	3735.51±18.40	4562.06±8.89	45.12±0.03	18920.08±52.33	4152.62±17.95	3367.54±74.07	4533.19±14.67	45.20±0.08	3890.58±26.33
TimesFM	20930.94±169.11	3796.55±55.78	4575.70±14.62	44.92±0.06	18980.20±258.82	4023.10±9.21	3904.39±25.80	4660.46±6.13	44.39±0.02	3632.71±14.58
Time-MoE	20792.74±23.40	3715.27±36.98	4551.60±13.40	45.16±0.04	19022.33±78.85	4089.13±13.52	3481.59±87.80	4585.52±17.57	45.06±0.09	3810.90±27.48
TimePrism	20736.17±7.78	3722.57±1.60	4551.46±0.93	45.10±0.01	18961.57±10.65	3787.82±25.40	4175.60±11.86	4725.36±9.75	43.26±0.07	3269.73±33.26
AgACI	22405.28±2.26	2910.65±2.47	4374.10±0.08	45.79±0.01	21348.47±3.30	4684.79±0.39	2451.09±0.04	2605.31±0.32	46.81±0.00	4655.52±0.36
Conformal PID	22110.78±15.47	3131.98±8.58	4424.31±2.47	45.65±0.01	20905.66±20.96	4711.00±0.09	2447.54±0.02	2570.00±0.21	46.86±0.00	4688.71±0.10
SAOCP	22123.13±2.10	3157.59±1.34	4429.23±1.36	45.61±0.01	20899.63±2.67	4718.07±0.14	2447.47±0.04	2569.08±0.28	46.85±0.00	4696.10±0.15
COP	22103.72±12.10	3137.27±5.62	4424.44±2.32	45.64±0.01	20895.53±15.85	4710.35±0.09	2447.75±0.01	2573.13±0.10	46.85±0.00	4687.87±0.11
DeCal (Ours)	**22663.65**±0.20	**2659.59**±0.11	**4329.73**±0.75	**45.96**±0.00	**21748.68**±0.64	**4746.49**±0.09	**2446.68**±0.01	**2557.35**±0.12	**46.88**±0.00	**4727.55**±0.08

Workloads. We use two complementary public LLM-serving workload sources. ServeGen [24] is the main multi-client workload family. We report the language m-mid setting, which has 92 clients, 54 timestamps, and 2,840 adjacent-window pairs after preprocessing. Its mean token length is 2,361.1, with P95 6,435.7 and P99 11,615.7. In adjacent transitions, 42.2% satisfy JS $\geq$ 0.1 and 17.7% satisfy JS $\geq$ 0.3. BurstGPT [22] provides an independent production-traffic workload. It is aggregated into four streams, each treated as a budget entity, and contains 4,674 timestamps and 11,925 adjacent-window pairs. Its mean token length is 957.5, with P95 2,459.2 and P99 3,206.1. In adjacent transitions, 89.4% satisfy JS $\geq$ 0.1 and 82.1% satisfy JS $\geq$ 0.3. ServeGen stresses multi-client heterogeneity and transfer structure, whereas BurstGPT stresses high-frequency regime shifts and online calibration delay. Appendix D reports workload diagnostics that make this contrast explicit.

Preprocessing. Both workload sources are converted into budget entities, time windows, and empirical token-length distributions. ServeGen requests are aggregated by client identifier and timestamp, and BurstGPT requests are aggregated by stream. Each window stores a normalized one-token-resolution histogram $P_{c,t}$, from which P95, P99, P99.9, mean, request count, and request-weighted quantities are derived. Windows with fewer than `min_requests=10` are dropped. Drift labels are computed offline from adjacent-window $\mathrm{JS}(P_{c,t-1}, P_{c,t})$ only for evaluation and event alignment, and are never exposed before a budget decision.

Metric scope. All budget decisions and realized tail demands are expressed in tokens. The direct metrics report P99 Coverage, request-weighted tail miss mass, over-allocation, waste, and absolute percentage error, while trigger behavior and KV-cache pressure analysis evaluate mechanism behavior and downstream capacity utility. The throughput, latency, Quality Proxy, and SLO Goodput columns in Table 1 are replay-derived indicators computed after budget emission from the same emitted budgets and realized histograms. They summarize capacity pressure under a common evaluator and should not be interpreted as measurements from a physical serving engine or as task-accuracy benchmarks.

Baselines. Table 1 compares DeCal with 12 adapted reference methods under the same completed-history budget task. The baselines represent four design assumptions rather than identical end-to-end serving systems. *LLM serving length prediction* methods, VLLM-LTR [8] and EGTP [25], originate from request-level prediction or ranking tasks. Their wrappers remove current-request or current-batch signals and test whether the retained prediction principle transfers to completed-history budgeting. *Workload forecasting* methods, Fremer [26], OrgLinear [7], and PRISM [23], extrapolate tail statistics from historical workload time series. *Time-series foundation models*, TimesFM [6], Time-MoE [19], and TimePrism [5], test generic sequence forecasting under the same causal input boundary. *Online conformal* methods, AgACI [27], Conformal PID [2], SAOCP [3], and COP [11], are the closest task-level references because they also adjust budgets through historical error feedback. Table 2 complements this comparison with controlled variants of DeCal's base-budget scoring, protection path, score reference, and optional transfer prior. Every wrapper receives only completed histories, emits one scalar $B_{c,t}$ in the same token coordinate, and is scored by the same evaluator. Drift labels and realized current-window quantiles are used only for offline reporting, not for budget decisions. Appendix C.1 records the adaptation boundary and clarifies that cross-task wrappers provide contextual references rather than direct claims over the original systems.

Table 2. Component ablation matrix for base-budget scoring, drift-triggered protection, and selective transfer. Check marks indicate enabled components.

Index	Base-Budget Scoring	Drift-Triggered Protection	Selective Transfer	ServeGen			BurstGPT		
				Waste ↓	MAPE ↓	P99 Coverage ↑	Waste ↓	MAPE ↓	P99 Coverage ↑
A	✗	✗	✗	0.2821	4.5920	0.6006	0.5476	5.6418	0.9089
B	✓	✗	✗	0.2713	3.2784	0.7384	0.4648	4.8596	0.9085
C	✓	✓	✗	0.2488	3.4195	0.8431	0.4646	4.8583	0.8765
D	✗	✓	✓	0.1974	3.4050	0.8568	0.4189	4.2740	0.8359
E	✓	✓	✓	**0.1895**	**2.8966**	**0.8636**	**0.4124**	**3.6865**	**0.9453**

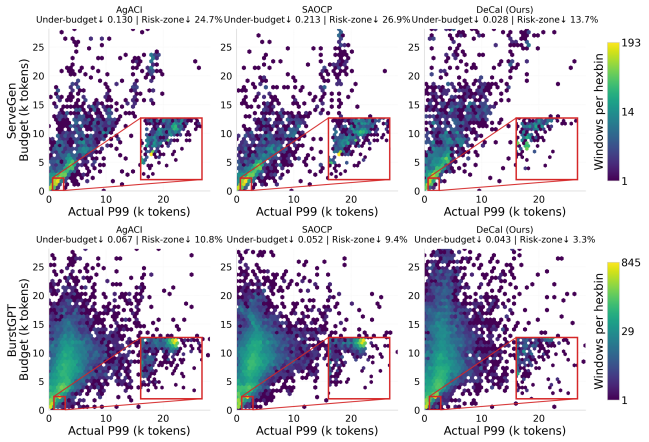


Figure 8. Budget-demand alignment under completed-history replay. Each cell aggregates budget-entity windows by emitted budget and realized next-window P99. The diagonal indicates exact agreement in the shared token coordinate.

4.2 Overall Performance (RQ1)

Table 1 compares all methods under the same completed-history replay interface. Each method emits a per-window tail-length proxy budget, and the fixed evaluator maps that sequence into throughput, latency, Quality Proxy, and SLO Goodput indicators. Prediction and forecasting baselines estimate historical trends but do not combine a drift-triggered protection state with a separate base-budget residual reference. Online conformal methods provide historical error feedback, but they still rely on failures or large residuals entering the buffer after drift.

Under the fixed replay mapping, DeCal improves the reported capacity-pressure trade-off without globally inflating budgets. On ServeGen, throughput increases from AgACI's 22,405.28 to 22,663.65 token/s, P95 latency decreases from 2,910.65 ms to 2,659.59 ms, and SLO Goodput increases from 21,348.47 to 21,748.68 token/s. On BurstGPT, DeCal obtains the best value across throughput, P95/P99 latency, Quality Proxy, and SLO Goodput. Because these columns are derived from one evaluator and one budget sequence, their agreement indicates a consistent replay-level trade-off rather than independent measurements of system performance. Fig. 8 explains the aggregate result at the budget-signal level for AgACI, SAOCP, and DeCal. Useful budgets should concentrate near the diagonal and avoid mass below it, where realized P99 demand exceeds the committed budget. Mass far above the diagonal indicates excess reservation. DeCal reduces under-budgeted mass relative to the displayed online-calibration references while retaining substantial density near the diagonal. This pattern is consistent with the capacity-pressure indicators in Table 1.

4.3 Ablation Study (RQ2)

The component ablation compares base-budget scoring, drift-triggered protection, and the optional transfer prior under the same replay and metric definitions. Index A has no base-budget score reference, protection, or transfer and reaches 0.6006 ServeGen P99 Coverage. Enabling only base-budget scoring in Index B raises coverage to 0.7384, indicating that the residual learner benefits from a stable score reference before protection is added. Adding protection without transfer in Index C raises coverage to 0.8431. The two mechanisms operate on different time scales. Base-budget scoring preserves the long-term residual target, while protection covers the interval before enough post-drift scores enter the buffer.

Indices D and E keep the trigger, protection, hold-window, and transfer settings fixed and change only the score reference. Index D records the residual against the full emitted action, including both protection and the existing correction, whereas Index E records the residual against the base budget. On BurstGPT, the full-budget reference reaches 0.8359 P99 Coverage, whereas the base-budget reference reaches 0.9453. The comparison supports the action–reference separation used by DeCal. Because the reference change removes both the protection term and the existing correction, the ablation does not attribute the entire gap to either term in isolation.

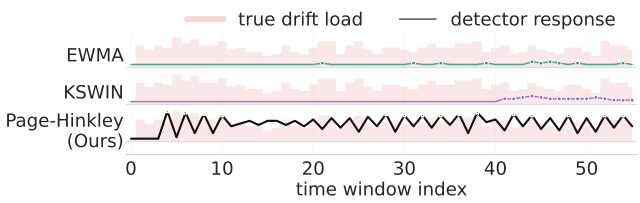


Figure 9. Qualitative completed-window drift diagnostic. The shaded trace is offline drift load, and the lines are normalized detector responses. Marked Page–Hinkley peaks denote trigger activations. The figure illustrates timing behavior rather than detection accuracy.

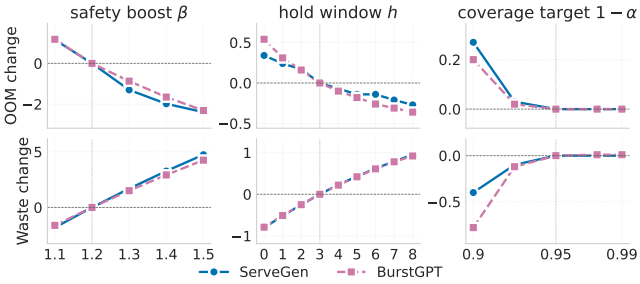


Figure 10. Sensitivity sweep over protection coefficient, hold length, and coverage target.

4.4 Drift-Trigger Behavior (RQ3)

The trigger diagnostic compares Page–Hinkley with EWMA and KSWIN responses on the same completed-window sequence. Its purpose is not to rank detectors as standalone classifiers or establish detection accuracy. Instead, it visualizes when a low-cost timing signal becomes active as the completed workload changes. Fig. 9 overlays offline drift load with normalized response traces. The figure is qualitative and does not report detection delay, false-alarm rate, or downstream miss reduction. On this sequence, Page–Hinkley remains active through the high-variation region and activates before the displayed KSWIN response. The observation is descriptive rather than a general detector ranking. Its role is limited to timing and does not determine the budget magnitude or the calibration target. Because frequent alarms can increase protection cost, DeCal bounds the protection coefficient, holds it for a short interval, and then retracts it rather than treating it as a permanent reserve.

4.5 Operating Sensitivity and Cost (RQ4)

The sensitivity study varies the protection-coefficient boost, hold length, and coverage target around the evaluated operating point. It evaluates how the controller moves along the budget-miss–waste frontier.

Fig. 10 indicates that protection-path parameters have the clearest effect. Larger boosts or longer holds reduce underbudgeting but increase waste, while smaller boosts or shorter

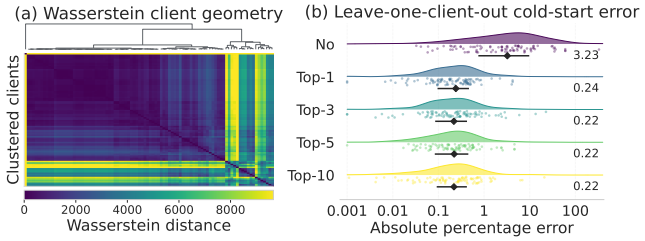


Figure 11. Cold-start transfer on ServeGen. The left panel shows the leave-one-client-out nearest-donor geometry. The right panel reports startup APE as the donor count k increases.

holds reclaim budget at the cost of more tail failures. Raising the coverage target has limited effect after the correction quantile reaches the upper end of the finite buffer. Across these settings, the score reference remains the base budget. Appendix D evaluates JS threshold, window granularity, minimum-request filters, adaptive-quantile hysteresis, and paired-bootstrap uncertainty.

Cold-start transfer. Leave-one-client-out cold start on ServeGen evaluates whether selective transfer helps when local history is thin. For each target client, the target is removed from the donor pool, one local reference distribution is exposed, and the prior is constructed from the top-k donors using only completed histories. Future target-client information is excluded. The protocol tests whether the similarity structure in Section 2.2 provides a causal startup prior without replacing local history. Fig. 11(a) illustrates the resulting nearest-neighbor geometry rather than a globally pooled state. Fig. 11(b) shows that aggregate absolute percentage error (APE) decreases from 3.23 without transfer to 0.24 with the nearest donor and 0.22 with three donors. The value remains 0.22 with five or ten donors, so most of the gain comes from a small set of nearby clients. The result supports bounded selective transfer as a base-estimator prior while keeping normal updates local and decoupled. BurstGPT contains only four streams, so leave-one-stream transfer does not provide sufficient evidence for the same analysis.

DeCal updates once per window, and its normal update path excludes transfer unless startup or severe-drift recovery activates the transfer prior. Section 3.4 defines the transfer construction and activation conditions, and Appendix B gives the online-cost details.

4.6 Capacity-Signal Utility (RQ5)

KV-cache pressure bridge. We convert each emitted tail-length proxy budget into KV-cache bytes using the fixed geometry in Appendix C.2. The converted budgets are then aggregated across entities at each timestamp. Fig. 12 reports the resulting shared-pool pressure. A 1x shared pool still overflows in 50.0% of windows, while a 2x pool reduces overflow to 17.7% with 53.6% mean utilization. Because the traces

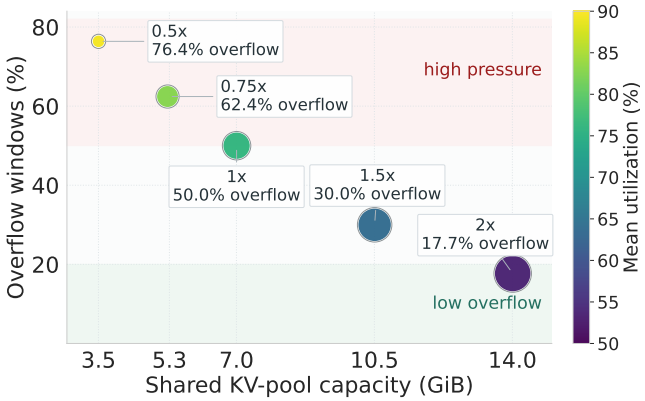


Figure 12. KV-cache shared-pool pressure induced by emitted budgets. The capacity multiplier is swept on the x-axis. Plotted quantities are overflow rate and utilization.

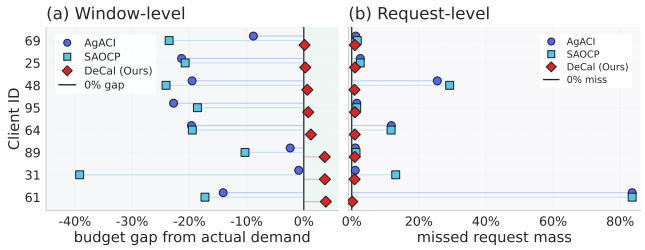


Figure 13. Selected-entity budget gaps and missed request mass for ServeGen clients. The left panel reports the gap between emitted budget and realized demand. The right panel reports request mass above the emitted budget.

do not include physical KV-block logs, this analysis evaluates budget-derived capacity pressure rather than end-to-end engine behavior. The result indicates that the budget sequence can serve as an auditable signal for admission control, quota pressure, or autoscaling.

Selected-entity behavior. Aggregate means can hide entities for which the committed budget remains below realized tail demand. Fig. 13 reports window-level budget gaps and request-level missed mass for selected ServeGen entities. Negative gaps and positive missed mass identify uncovered demand. DeCal keeps the displayed gaps at or above zero and the missed request mass near zero more consistently than AgACI and SAOCP. The reference methods leave substantial demand uncovered for several entities. The selected-entity view complements the aggregate table by showing how the upstream budget signal changes uncovered demand at the entity level.

5 Discussion

The evaluation clarifies why the calibration reference is central under completed-history visibility. A temporary budget increase can protect the current window, but using the protected budget as the later score reference changes what the calibration buffer learns. DeCal avoids this effect by letting protection act on the emitted budget while calibration records the residual against the base budget. The component comparison supports this separation, and the selected-entity results show that the same design choice remains visible at the entity level. The trigger analysis further explains when protection becomes active, but it is not intended as a detector-accuracy claim.

The evidence should be interpreted as an evaluation of the upstream budget signal. All methods obey the same visibility boundary, emit a budget before the target window is observed, and are scored by the same replay evaluator after the window closes. Within this setting, selective transfer is supported as a bounded startup or recovery prior for the base estimator. The results do not establish global pooling as a general strategy, and they do not substitute for execution-layer evaluation. The KV-cache pressure bridge shows that emitted token budgets can be mapped to capacity pressure under fixed model geometry, which connects the budget signal to admission, quota, and scaling decisions.

Limitations. The replay-derived indicators in Table 1 are not execution-layer measurements. The adapted request-level baselines are controlled task references, not reproductions of the original systems. BurstGPT has only four streams, which limits conclusions about cross-entity transfer on that workload. The trigger analysis evaluates timing behavior rather than detector accuracy. Future work can integrate the upstream budget signal with execution-layer KV-block logs and downstream admission, quota, or autoscaling policies to study deployment-level effects.

6 Conclusion

This paper studies tail budgeting for non-stationary LLM-serving workloads under completed-history visibility. The key challenge is that capacity must be committed before the next tail state is observed, while feedback arrives only after the window closes. DeCal addresses this delayed-feedback setting by separating temporary protection in the emitted action from the base-budget reference used for calibration. Trace-driven replay on ServeGen and BurstGPT shows that this separation improves P99 Coverage and replay-derived capacity-pressure indicators relative to the evaluated variants. The cold-start and KV-cache pressure analyses further show that bounded transfer and token-to-capacity mapping make the emitted budget useful as an auditable upstream signal. DeCal therefore provides a causal tail budget controller that protects early post-drift windows without allowing temporary protection to contaminate later calibration.

References

[1] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav S. Gulavani, Ramachandran Ramjee, and Alexey Tumanov. 2024. VIDUR: A LARGE-SCALE SIMULATION FRAMEWORK

FOR LLM INFERENCE. In *Proceedings of the 7th Machine Learning and Systems*, Vol. 6. 351–366.

[2] Anastasios Angelopoulos, Emmanuel Candes, and Ryan J Tibshirani. 2023. Conformal PID Control for Time Series Prediction. In *Proceedings of the 37th Advances in Neural Information Processing Systems*, Vol. 36. 23047–23074.

[3] Aadyot Bhatnagar, Huan Wang, Caiming Xiong, and Yu Bai. 2023. Improved Online Conformal Prediction via Strongly Adaptive Online Learning. In *Proceedings of the 40th International Conference on Machine Learning*, Vol. 202. 2337–2363.

[4] Albert Bifet and Ricard Gavalda. 2007. Learning from time-changing data with adaptive windowing. In *Proceedings of the 7th SIAM international conference on data mining*. 443–448.

[5] Xilin Dai, Zhijian Xu, Wanxu Cai, and Qiang Xu. 2026. From Samples to Scenarios: A New Paradigm for Probabilistic Forecasting. In *Proceedings of the 14th International Conference on Learning Representations*.

[6] Abhimanyu Das, Weihao Kong, Rajat Sen, and Yichen Zhou. 2024. A decoder-only foundation model for time-series forecasting. In *Proceedings of the 41st International Conference on Machine Learning*. Article 404, 20 pages.

[7] Jiaang Duan, Shenglin Xu, Shiyou Qian, Dingyu Yang, Kangjin Wang, Chenzhi Liao, Yinghao Yu, Qin Hua, Hanwen Hu, Qi Wang, Wenchao Wu, Dongqing Bao, Tianyu Lu, Jian Cao, Guangtao Xue, Guodong Yang, Liping Zhang, and Gang Chen. 2026. GFS: A Preemption-aware Scheduling Framework for GPU Clusters with Predictive Spot Instance Management. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 117–131.

[8] Yichao Fu, Siqi Zhu, Runlong Su, Aurick Qiao, Ion Stoica, and Hao Zhang. 2024. Efficient LLM Scheduling by Learning to Rank. In *Proceedings of the 38th Advances in Neural Information Processing Systems*, Vol. 37. 59006–59029.

[9] Isaac Gibbs and Emmanuel Candes. 2021. Adaptive Conformal Inference Under Distribution Shift. In *Proceedings of the 35th Advances in Neural Information Processing Systems*, Vol. 34. 1660–1672.

[10] Isaac Gibbs and Emmanuel J. Candès. 2024. Conformal Inference for Online Prediction with Arbitrary Distribution Shifts. *Journal of Machine Learning Research* 25, 162 (2024), 1–36.

[11] Dongjian Hu, Junxi Wu, Shu-Tao Xia, and Changliang Zou. 2026. Distribution-informed Online Conformal Prediction. In *Proceedings of the 14th International Conference on Learning Representations*.

[12] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.

[13] E. S. Page. 1954. Continuous Inspection Schemes. *Biometrika* 41, 1/2 (1954), 100–115.

[14] E. S. Page. 1955. A Test for a Change in a Parameter Occurring at an Unknown Point. *Biometrika* 42, 3/4 (1955), 523–527.

[15] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture*. 118–132.

[16] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Heyi Tang, Feng Ren, Teng Ma, Shangming Cai, Yineng Zhang, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving. *ACM Transactions on Storage* (2025).

[17] Jing Ren, Jiapeng Du, Bowen Li, Ziqi Xu, Xin Zheng, Hong Jia, Suyu Ma, Xiwei Xu, and Feng Xia. 2026. Spiking Graph Predictive Coding for Reliable OOD Generalization. In *Proceedings of the 35th ACM Web Conference*. 9056–9067.

[18] Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E. Gonzalez, and Ion Stoica. 2024. Fairness in Serving Large Language Models. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation*. 965–988.

[19] Xiaoming Shi, Shiyu Wang, Yuqi Nie, Dianqi Li, Zhou Ye, Qingsong Wen, and Ming Jin. 2025. Time-MoE: Billion-Scale Time Series Foundation Models with Mixture of Experts. In *Proceedings of the 13th International Conference on Learning Representations*, Vol. 2025. 34635–34667.

[20] Vikranth Srivatsa, Zijian He, Reyna Abhyankar, Dongming Li, and Yiying Zhang. 2025. Preble: Efficient Distributed Prompt Scheduling for LLM Serving. In *Proceedings of the 13th International Conference on Learning Representations*, Vol. 2025. 37057–37082.

[21] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llumnix: Dynamic Scheduling for Large Language Model Serving. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation*. 173–191.

[22] Yuxin Wang, Yuhan Chen, Zeyu Li, Xueze Kang, Yuchu Fang, Yeju Zhou, Yang Zheng, Zhenheng Tang, Xin He, Rui Guo, Xin Wang, Qiang Wang, Amelie Chi Zhou, and Xiaowen Chu. 2025. BurstGPT: A Real-World Workload Dataset to Optimize LLM Serving Systems. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*. 5831–5841.

[23] Xin Wu, Fei Teng, Xingwang Li, Bin Zheng, and Qiang Duan. 2026. PRISM: Dynamic Primitive-Based Forecasting for Large-Scale GPU Cluster Workloads. In *Proceedings of the 63rd ACM/IEEE Design Automation Conference*.

[24] Yuxing Xiang, Xue Li, Kun Qian, Yan Zhang, Wenyuan Yu, Ennan Zhai, Xin Jin, and Jingren Zhou. 2026. ServeGen: Workload Characterization and Generation of Large Language Model Serving in Production. In *Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation*. 1845–1859.

[25] Huanyi Xie, Yubin Chen, Liangyu Wang, Lijie Hu, and Di Wang. 2026. Predicting LLM Output Length via Entropy-Guided Representations. In *Proceedings of the 14th International Conference on Learning Representations*, Vol. 2026.

[26] Hengyu Ye, Jiadong Chen, Fuxin Jiang, Xiao He, Tieying Zhang, Jianjun Chen, and Xiaofeng Gao. 2025. Fremer: Lightweight and Effective Frequency Transformer for Workload Forecasting in Cloud Services. *Proceedings of the VLDB Endowment* 18, 11 (2025), 3812–3825.

[27] Margaux Zaffran, Olivier Feron, Yannig Goude, Julie Josse, and Aymeric Dieuleveut. 2022. Adaptive Conformal Predictions for Time Series. In *Proceedings of the 39th International Conference on Machine Learning*, Vol. 162. 25834–25866.

[28] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation*. 193–210.

A Related Work

Prior work shows that LLM-serving demand varies substantially over time and across serving entities. BurstGPT characterizes temporal variation in production traffic [22], ServeGen enables controlled replay of multi-entity workloads [24], and Vidur connects conversation traces with simulation-based serving evaluation [1]. A separate line of serving-system work focuses on allocating or scheduling capacity after demand has become visible. vLLM reduces KV-cache fragmentation through paged allocation [12]. DistServe and Splitwise separate prefill and decode pressure [15, 28], while

Llumnix supports dynamic scheduling [21]. Mooncake extends capacity through KV-centric offloading [16], and Preble exploits prefix reuse to reduce redundant computation and cache pressure [20]. DeCal addresses a different point in the control path by committing an upstream tail budget before the next workload window is observed.

Forecasting methods provide a natural way to derive a next-window capacity signal from historical demand. At the request level, EGTP predicts output length from internal uncertainty signals [25], and vLLM-LTR uses length ranking for serving decisions [8]. At the workload level, Fremer models dynamics in the frequency domain [26], OrgLinear forecasts organization-level GPU demand [7], and PRISM combines decomposition with spectral refinement [23]. Time-series foundation models extend this direction through decoder-only forecasting [6], mixture-of-experts architectures [19], and probabilistic scenario generation [5]. Such predictors can replace or improve DeCal's base estimator, but they do not by themselves define how temporary protection should enter later calibration. DeCal is therefore complementary to advances in predictive accuracy.

Online calibration and drift detection are closest to DeCal's feedback structure. Online conformal methods adapt after miscoverage [9], aggregate multiple update rates [27], introduce control-inspired corrections [2], provide strongly adaptive updates [10], or estimate score distributions [11]. Drift detectors provide timing information through adaptive windows [4], cumulative change statistics [13], or lightweight mean-shift tests [14]. Calibration determines the correction magnitude after errors enter the history, whereas detection indicates when a change may have occurred. The two roles remain coupled if both update against the same temporarily protected budget. DeCal separates current-window protection from long-term residual learning by assigning them distinct budget references.

B Transfer Prior and Online Complexity

Section 3.4 specifies the transfer activation rule, donor construction, and leakage boundary. The remaining cost follows directly from the per-window update. For each entity, the trigger keeps constant-size state and updates in $O(1)$ time. Updating an M-bin distribution estimate requires $O(M)$ time and space. The calibration buffer stores the latest R scores, and direct empirical-quantile computation costs $O(R \log R)$. An order-statistics structure can reduce the quantile update to $O(\log R)$. When transfer is active, nearest-neighbor search over N committed entity states costs $O(NM)$. Since transfer is used only during cold start or severe-drift recovery, it is outside the normal per-window update path.

This accounting is used in the evaluation when interpreting online overhead and cold-start transfer.

C Evaluation Protocol and Metrics

C.1 Baseline Adaptation Protocol

All reference methods are evaluated through the same causal interface. Before window t is revealed, a method receives completed per-entity histories and emits one scalar budget $B_{c,t}$. The request-level wrappers for EGTP and VLLM-LTR remove current-request and current-batch inputs. Workload forecasters and time-series models consume completed tail-statistic sequences and produce one-step forecasts. Online conformal controllers use the same base sequence and update only after $P_{c,t}$ is observed, making them the closest task-level references. All methods share preprocessing, target windows, replay order, evaluator, and the same prohibition on current-window or future information.

Table 3 summarizes the adaptations. Every method targets $y_{c,t} = q_{0.99}(P_{c,t})$. Prediction and forecasting methods emit point estimates, whereas online conformal methods apply method-specific corrections to a shared EWMA P99 forecast. The conformal methods and DeCal use the same target coverage, $\tau = 0.95$. Neural wrappers share a 32-window history, a common training schedule, and the same replay split. No method uses dataset-specific hyperparameter search.

The protocol standardizes the information boundary, target variable, and evaluator. It does not make cross-task wrappers equivalent to the original end-to-end systems, nor does it require all method families to achieve identical realized coverage. Prediction and forecasting methods therefore serve as controlled contextual references. Online conformal methods provide the closest task-level comparison because they convert historical P99 errors into next-window budget corrections.

The capacity-pressure evaluator is applied after budget emission using the emitted budgets and realized histograms. For each window, it computes P99 budget miss, request-weighted tail mass above the budget, over-allocation, and KV-byte demand. The same budget sequence is then mapped to throughput, latency, SLO Goodput, and Quality Proxy under the fixed replay evaluator used in Table 1. Table 1 therefore compares budget signals under a common replay mapping. Its columns are derived views of each budget trajectory rather than independent execution-layer measurements.

C.2 Replay Metric Definitions

The primary window-level proxy risk metrics are the P99 budget-miss rate m_{99} and P99 Coverage c_{99}.

$$\begin{aligned} m_{99} &= \frac{1}{N} \sum_{(c,t)\in\mathcal{I}} \mathbf{1}[y_{c,t} > B_{c,t}], \\ c_{99} &= \frac{1}{N} \sum_{(c,t)\in\mathcal{I}} \mathbf{1}[B_{c,t} \geq y_{c,t}] = 1 - m_{99}, \end{aligned} \tag{13}$$

Here, $\mathcal{I}$ denotes the evaluated windows and $N = |\mathcal{I}|$. A

Table 3. Baseline adaptation to next-window P99 budgeting.

Method	Causal adaptation and budget output	Key configuration
VLLM-LTR	Binned predictor trained on completed P99 histories. Current-request features are removed.	32 bins, width 64
EGTP	Binned predictor trained on completed P99 histories. Hidden-state inputs are removed.	32 bins, width 16
Fremer	Frequency refinement trained for causal one-step P99 forecasting.	Cutoff 2, width 32
OrgLinear	Historical trend decomposition for P99 forecasting. Organization and calendar features are removed.	Kernel 5, width 32
PRISM	Historical sequence decomposition for P99 forecasting.	Patch 4, stride 2, width 32, 8 components
TimesFM	Compact sequence forecaster trained from scratch on P99 histories.	Patch 4, width 64, 2 layers
Time-MoE	Compact expert-based forecaster trained from scratch on P99 histories.	Width 48, 2 layers, 4 experts
TimePrism	Historical trend decomposition for P99 forecasting. Scenario output is removed.	Width 64, kernel 5
AgACI	Adaptive online correction over the shared P99 base forecast.	Coverage 0.95, window 256
Conformal PID	Control-inspired online correction over the shared P99 base forecast.	Coverage 0.95, burn-in 50
SAOCP	Strongly adaptive online correction over the shared P99 base forecast.	Coverage 0.95, base lifetime 8, adaptive scale
COP	Distribution-informed online correction over the shared P99 base forecast.	Coverage 0.95, window 50, scale 0.5

P99 budget miss is defined with respect to the upstream budget signal, not as a physical engine-level out-of-memory event. Because the window-level metric weights all windows equally, we also report the request-weighted tail mass above the budget.

$$m_{\text{req}} = \frac{\sum_{(c,t)\in\mathcal{I}} w_{c,t} \sum_{\ell > B_{c,t}} p_{c,t}(\ell)}{\sum_{(c,t)\in\mathcal{I}} w_{c,t}}. \tag{14}$$

Resource cost is summarized by over-allocation and normalized waste.

$$\begin{aligned} \text{OverAlloc} &= \frac{1}{N} \sum_{(c,t)\in\mathcal{I}} [B_{c,t} - y_{c,t}]_+, \\ \text{Waste} &= \frac{1}{N} \sum_{(c,t)\in\mathcal{I}} \frac{[B_{c,t} - y_{c,t}]_+}{\max(B_{c,t}, \epsilon)}. \end{aligned} \tag{15}$$

Per-window absolute percentage error summarizes budget accuracy.

$$\text{APE}_{c,t} = \frac{|B_{c,t} - y_{c,t}|}{\max(y_{c,t}, \epsilon)}. \tag{16}$$

MAPE is the mean of $\text{APE}_{c,t}$ over the evaluated windows. The KV bridge converts tail-length proxy budgets to bytes under fixed model geometry.

$$\text{KVBytes}_{c,t} = B_{c,t} \cdot 2LH_{kv}d \cdot b. \tag{17}$$

For the bridge experiments, $L = 32$, $H_{kv} = 32$, $d = 128$, and FP16, yielding $2LH_{kv}db = 524{,}288$ bytes/token.

C.3 KV-Cache Pressure Bridge

The KV-cache pressure bridge evaluates whether tail-length proxy budgets provide an interpretable downstream capacity signal. At each timestamp, budgets are aggregated across entities and converted from tokens to bytes using Appendix C.2. For shared-pool capacity C, we define

$$\begin{aligned} \text{overflow}_t(C) &= \mathbf{1}\left[\sum_c B_{c,t} \cdot 524{,}288 > C\right], \\ \text{util}_t(C) &= \min\left(1, \frac{\sum_c B_{c,t} \cdot 524{,}288}{C}\right). \end{aligned} \tag{18}$$

The capacity sweep reports overflow rate and mean utilization. It does not model a physical scheduler. Instead, it evaluates whether the upstream budget can serve as a downstream capacity-pressure signal. Fig. 12 presents the sweep in the main text.

The sweep shows the expected capacity trade-off. A 1x pool overflows in 50.0% of windows. Increasing capacity to 2x reduces overflow to 17.7% and mean utilization to 53.6%. Tighter capacity improves utilization but increases pressure on admission, quota, or scaling. Looser capacity reduces overflow at the cost of additional reserved memory. All values are derived from causally emitted budgets rather than hindsight P99 demand.

The bridge measures budget-derived pressure rather than end-to-end engine behavior. Under fixed model geometry, tail-length budgets map linearly to KV bytes. Overflow and utilization can therefore serve as auditable inputs for autoscaling, quota monitoring, and admission control.

D Robustness and Sensitivity Analysis

This section examines whether DeCal's conclusions persist under a stronger heuristic baseline, alternative uncertainty blocks, drift-threshold changes, and preprocessing variants. All experiments preserve the replay order, visibility boundary, and post-emission metrics defined in Appendix C.

D.1 Adaptive-Quantile Baseline

Fig. 14 compares DeCal with EWMA and an adaptive-quantile hysteresis baseline using P99 budget-miss rate and replay-derived SLO Goodput. The hysteresis baseline already adapts its quantile target and retracts protection over time. DeCal improves both metrics on ServeGen and BurstGPT, indicating that the gains are not explained by smoothing or hysteresis alone.

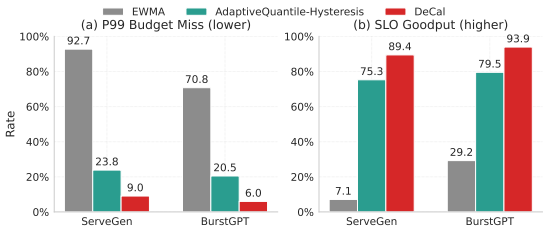


Figure 14. Reliability and replay-derived SLO Goodput under the adaptive-quantile baseline comparison.

D.2 Paired Bootstrap Uncertainty Analysis

Fig. 15 reports paired block-bootstrap uncertainty for the comparison with adaptive-quantile hysteresis. The plotted quantity is confidence-interval width rather than the effect estimate. The figure should therefore be interpreted as an uncertainty diagnostic, not as a test of whether an effect crosses zero. Entity and timestamp blocking produce different widths, particularly on ServeGen, which indicates sensitivity to the assumed correlation unit. We do not use this diagnostic to claim statistical significance for small differences.

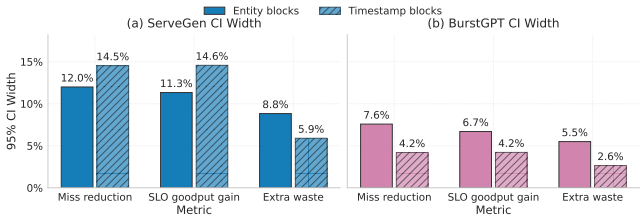


Figure 15. Paired block-bootstrap uncertainty of DeCal improvements over adaptive-quantile hysteresis.

D.3 Drift Threshold and Preprocessing Sensitivity

Fig. 16 varies the Jensen–Shannon divergence threshold used to label drift windows. As expected, the threshold changes the number of selected windows, particularly for ServeGen. DeCal nevertheless retains its relative advantage within the resulting drift subsets.

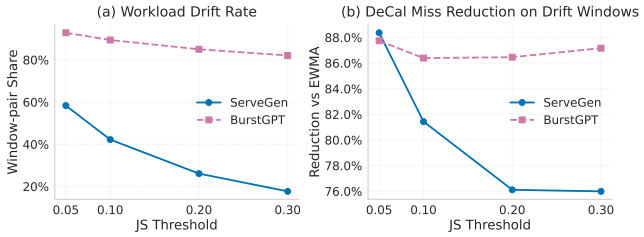


Figure 16. Sensitivity of drift-window conclusions across JS-divergence thresholds.

Fig. 17 varies window granularity and the minimum request count. Coarser aggregation changes the absolute miss–waste operating point, especially on ServeGen. Increasing the request-count threshold has little effect on DeCal's relative miss reduction over EWMA. The observed advantage is therefore not driven solely by small windows with unstable empirical quantiles.

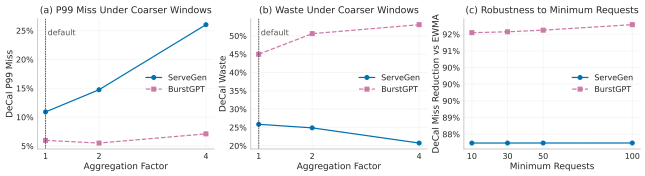


Figure 17. Sensitivity of DeCal to window granularity and minimum-request filtering.

D.4 Workload Diagnostics

Fig. 18 summarizes request counts and divergence profiles for both datasets. ServeGen emphasizes cross-client heterogeneity, whereas BurstGPT emphasizes repeated temporal shifts across a small set of streams. Together, the traces test multi-client heterogeneity and persistent drift under the same completed-history visibility constraint. BurstGPT's four streams, however, limit the strength of cross-entity conclusions on that workload.

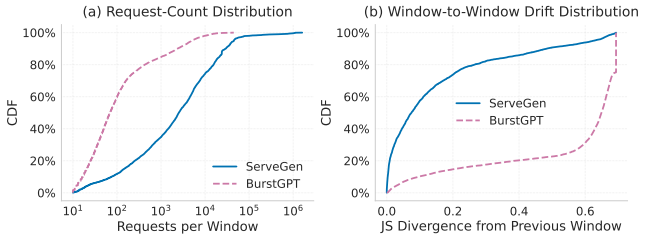


Figure 18. Workload heterogeneity and drift profiles of the two evaluation datasets.